

Mysql Join Types



Joins in Mysql

Reasoned about MySQL joins for 12 seconds >

In MySQL, "joins" are used to combine rows from two or more tables based on a related column between them. They're essential when your database is normalized and data is spread across multiple tables. Here's an overview of the most common join types:

1. INNER JOIN

- **Definition:** Returns records that have matching values in both tables.
- **Example:**

```
sql

SELECT a.column1, b.column2
FROM table1 AS a
INNER JOIN table2 AS b ON a.id = b.table1_id;
```

- **Usage:** Use an inner join when you only need records with matches in both tables.

2. LEFT JOIN (or LEFT OUTER JOIN)

- **Definition:** Returns all records from the left table, and the matched records from the right table. If there's no match, the result is NULL on the right side.
- **Example:**

```
sql

SELECT a.column1, b.column2
FROM table1 AS a
LEFT JOIN table2 AS b ON a.id = b.table1_id;
```

- **Usage:** Use a left join when you want all rows from the primary (left) table regardless of whether there's a corresponding match in the second table.

3. RIGHT JOIN (or RIGHT OUTER JOIN)

- **Definition:** Returns all records from the right table, and the matched records from the left table. If there's no match, the result is NULL on the left side.
- **Example:**

```
sql

SELECT a.column1, b.column2
FROM table1 AS a
RIGHT JOIN table2 AS b ON a.id = b.table1_id;
```

- **Usage:** Use a right join when you need all rows from the second (right) table regardless of whether there's a corresponding match in the first table.

4. FULL OUTER JOIN

- **Definition:** Returns all records when there is a match in either left or right table.
- **Note:** MySQL does not support FULL OUTER JOIN directly. However, you can simulate it using a combination of LEFT JOIN and RIGHT JOIN with a UNION.
- **Example:**

```
sql

SELECT a.column1, b.column2
FROM table1 AS a
LEFT JOIN table2 AS b ON a.id = b.table1_id
UNION
SELECT a.column1, b.column2
FROM table1 AS a
RIGHT JOIN table2 AS b ON a.id = b.table1_id;
```

5. CROSS JOIN

- **Definition:** Returns the Cartesian product of both tables (i.e., every row from the first table is combined with every row from the second table).
- **Example:**

```
sql

SELECT a.column1, b.column2
FROM table1 AS a
CROSS JOIN table2 AS b;
```

- **Usage:** Use a cross join when you want all possible combinations of rows from two tables.

6. SELF JOIN

- **Definition:** A self join is when a table is joined with itself. This is useful when dealing with hierarchical data or comparing rows within the same table.
- **Example:**

```
sql

SELECT a.employee_name AS Manager, b.employee_name AS Subordinate
FROM employees AS a
INNER JOIN employees AS b ON a.id = b.manager_id;
```

Key Points to Remember

- **Choose the Right Join:** The type of join you use should depend on whether you want only matching records or all records from one of the tables.
- **Performance:** Joins can be resource-intensive on large datasets. Indexing join columns properly is important to optimize query performance.
- **Readability:** Use table aliases to make your queries more readable, especially when joining multiple tables.

Understanding these joins will help you write efficient SQL queries and design better database architectures. If you have any more questions or need further examples, feel free to ask!



guide me when to use join ,left join , right join based on the question

Reasoned for 11 seconds >

When deciding which type of join to use, consider what your main goal is—whether you need only matching records between tables or you want to retain all records from one table regardless of whether there's a match in the other.

1. INNER JOIN (a.k.a. JOIN)

- **When to Use:**

Use an **inner join** when your query should return only the records that have matching values in both tables.

- **Scenario Example:**

Suppose you have an `employees` table and a `departments` table. If you only want to list employees who are assigned to a department (i.e., there's a matching department record), then an inner join is appropriate.

- **SQL Example:**

```
sql

SELECT e.employee_name, d.department_name
FROM employees AS e
INNER JOIN departments AS d ON e.department_id = d.id;
```

2. LEFT JOIN (or LEFT OUTER JOIN)

- **When to Use:**

Use a **left join** when you need all the records from the left (first) table, even if there is no matching record in the right (second) table. Missing matches in the right table will appear as `NULL`.

- **Scenario Example:**

Imagine you want a list of all employees regardless of whether they have been assigned to a department. A left join on the `employees` table with the `departments` table ensures every employee appears in the result—even if some have no department information.

- **SQL Example:**

```
sql

SELECT e.employee_name, d.department_name
FROM employees AS e
LEFT JOIN departments AS d ON e.department_id = d.id;
```

3. RIGHT JOIN (or RIGHT OUTER JOIN)

- **When to Use:**

Use a **right join** when you need all the records from the right (second) table, even if there is no matching record in the left (first) table. Missing matches in the left table will appear as `NULL`.

- **Scenario Example:**

Consider you need a list of all departments, including those that currently have no employees assigned. In this case, you might right join the `departments` table to the `employees` table, so every department shows up regardless of employee assignment.

- **SQL Example:**

```
sql

SELECT e.employee_name, d.department_name
FROM employees AS e
RIGHT JOIN departments AS d ON e.department_id = d.id;
```

How to Choose Between LEFT and RIGHT JOIN

- **Primary Focus of Data:**

- If the main table of interest is on the left side of your query (for example, you want every employee listed), then use a left join.
- If the main focus is on the right side (for example, you want every department listed), then use a right join.

- **Interchangeability:**

Although left and right joins are functionally similar (you can often swap them by changing the order of tables), your decision should reflect which table is central to your analysis.

Quick Recap

- **INNER JOIN:** Use when you only want records that have a match in both tables.
- **LEFT JOIN:** Use when you want all records from the left table and only matching records from the right.
- **RIGHT JOIN:** Use when you want all records from the right table and only matching records from the left.

By aligning your join type with the focus of your query, you ensure that the results match your data analysis needs. Feel free to ask if you have a specific scenario or further questions!



```
use test;
CREATE TABLE salgrade(
  grade int(4) primary key,
  losal decimal(10,2),
  hisal decimal(10,2)
);

CREATE TABLE dept(
  deptno int(2) primary key,
  dname varchar(50) not null,
  location varchar(50) not null
```

```
);

CREATE TABLE emp(
  empno int(4) primary key,
  ename varchar(50) not null,
  job varchar(50) not null,
  mgr int(4),
  hiredate date,
  sal decimal(10,2),
  comm decimal(10,2),
  deptno int(2)
);

ALTER TABLE emp ADD CONSTRAINT fk_dept FOREIGN KEY (deptno)
REFERENCES dept (deptno);

# -----
DROP PROCEDURE IF EXISTS empCount;

# Get count of employees
CREATE PROCEDURE empCount (OUT param1 INT)
SELECT COUNT(*) INTO param1 FROM emp;

# calling/invoking the procedure
CALL empCount(@num);
select @num;

# -----
DROP PROCEDURE IF EXISTS empAverageSal;

# Get average salary of employees

DELIMITER $$
CREATE PROCEDURE empAverageSal(OUT avgSal decimal(10,2))
BEGIN
  SELECT AVG(sal) INTO avgSal FROM emp;
END $$

DELIMITER ;

call empAverageSal(@sal);
select @sal;

# -----
DROP PROCEDURE IF EXISTS getJob;

# Given empno, get the job
CREATE PROCEDURE getJob (IN param1 INT, OUT param2 varchar(50))
SELECT job into param2 from emp where empno = param1;

CALL getJob(7788, @job);
```

```

select @job;

# -----
DROP PROCEDURE IF EXISTS setCounter;

DELIMITER $$
CREATE PROCEDURE setCounter(INOUT count INT(3),IN inc INT(3))
BEGIN
    SET count = count + inc;
END$$
DELIMITER ;

SET @counter = 1;
CALL setCounter(@counter,1);
CALL setCounter(@counter,2);
CALL setCounter(@counter,3);
SELECT @counter;

# -----
DROP PROCEDURE IF EXISTS getEmpSalGrade;
DELIMITER $$

CREATE PROCEDURE getEmpSalGrade(
    in p_empNo int,
    out p_empSalGrade varchar(10))
BEGIN
    DECLARE empSal double;

    SELECT sal INTO empSal
    FROM emp
    WHERE empno = p_empNo;

    IF empSal > 3000 THEN
        SET p_empSalGrade = 'HIGH';
    ELSEIF (empSal > 1000 && empSal <= 3000 ) THEN
        SET p_empSalGrade = 'MEDIUM';
    ELSEIF (empSal <= 1000) THEN
        SET p_empSalGrade = 'LOW';
    END IF;

END$$

DELIMITER ;

call getEmpSalGrade(7839, @empSalGrade);
select @empSalGrade;

call getEmpSalGrade(7369, @empSalGrade);
select @empSalGrade;

call getEmpSalGrade(7698, @empSalGrade);

```

```

select @empSalGrade;

# -----
DROP PROCEDURE IF EXISTS getEmpSalGrade2;
DELIMITER $$

CREATE PROCEDURE getEmpSalGrade2(
    in p_empNo int,
    out p_empSalGrade varchar(10))
BEGIN
    DECLARE empSal double;

    SELECT sal INTO empSal
    FROM emp
    WHERE empno = p_empNo;

    CASE
    WHEN empSal > 3000 THEN
        SET p_empSalGrade = 'HIGH';
    WHEN (empSal > 1000 && empSal <= 3000 ) THEN
        SET p_empSalGrade = 'MEDIUM';
    WHEN (empSal <= 1000) THEN
        SET p_empSalGrade = 'LOW';
    END CASE;

END$$

DELIMITER ;

call getEmpSalGrade2(7839, @empSalGrade);
select @empSalGrade;

call getEmpSalGrade2(7369, @empSalGrade);
select @empSalGrade;

call getEmpSalGrade2(7698, @empSalGrade);
select @empSalGrade;

# -----
DROP PROCEDURE IF EXISTS setShippingDays;
DELIMITER $$

CREATE PROCEDURE setShippingDays(
    in p_deptNo int,
    out p_shippingDays varchar(50))
BEGIN
    DECLARE cLocation varchar(50);

    SELECT location INTO cLocation
    FROM dept
    WHERE deptno = p_deptNo;

```

```

CASE cLocation
when 'Dallas' THEN
  SET p_shippingDays = '1 day shipping';
when 'Tempe' THEN
  SET p_shippingDays = '2 days shipping';
when 'Chicago' THEN
  SET p_shippingDays = '3 days shipping';
ELSE
  SET p_shippingDays = '7 days shipping';
END CASE;

END$$

DELIMITER ;

call setShippingDays(20, @custShippingDays);
select @custShippingDays;

call setShippingDays(50, @custShippingDays);
select @custShippingDays;

call setShippingDays(30, @custShippingDays);
select @custShippingDays;

# -----

DROP PROCEDURE IF EXISTS initDB;
DELIMITER $$

CREATE PROCEDURE initDB()
BEGIN
  delete from emp;
  delete from dept;
  delete from salgrade;

END$$

DELIMITER ;

call initDB();

SHOW PROCEDURE STATUS WHERE Db = 'test';

# -----

```

Reasoned about MySQL procedures and tables for 7 seconds >

This script is an excellent example of how you can use stored procedures in MySQL to encapsulate business logic and make your SQL code modular and reusable. Let's break it down section by section:

1. Database and Table Setup

- **Database Selection:**

```
sql

use test;
```

This command tells MySQL to use the `test` database.

- **Table Creation:**

Three tables are created:

- **salgrade:** Contains salary grades with a lower and upper bound.
 - **dept:** Contains department information including department number, name, and location.
 - **emp:** Contains employee information with fields for employee number, name, job, manager, hire date, salary, commission, and department number.
- A foreign key constraint links the `emp.deptno` to `dept.deptno`.

2. Stored Procedures

a. empCount

- **Purpose:**
Returns the total number of employees.
- **How It Works:**
It selects the count of records from the `emp` table and stores it in an output parameter.
- **Usage:**

```
sql

CALL empCount(@num);
SELECT @num;
```

b. empAverageSal

- **Purpose:**
Returns the average salary of all employees.
- **How It Works:**
It calculates the average salary from the `emp` table and sets the result to an output parameter.
- **Usage:**

```
sql

CALL empAverageSal(@sal);
SELECT @sal;
```

c. getJob

- **Purpose:**
Retrieves the job title of a specific employee given their employee number.

- **How It Works:**

The procedure takes an input parameter (employee number) and returns the corresponding job title via an output parameter.

- **Usage:**

```
sql

CALL getJob(7788, @job);
SELECT @job;
```

d. setCounter

- **Purpose:**

Demonstrates the use of INOUT parameters by incrementing a counter.

- **How It Works:**

It takes the current counter value and an increment value, adds them, and updates the counter.

- **Usage:**

```
sql

SET @counter = 1;
CALL setCounter(@counter,1);
CALL setCounter(@counter,2);
CALL setCounter(@counter,3);
SELECT @counter;
```

e. getEmpSalGrade & getEmpSalGrade2

- **Purpose:**

Both procedures are designed to determine a salary grade ('HIGH', 'MEDIUM', or 'LOW') for an employee based on their salary.

- **Differences:**

- **getEmpSalGrade:** Uses an IF...ELSEIF structure.
- **getEmpSalGrade2:** Uses a CASE statement for the same logic.

- **Usage:**

```
sql

CALL getEmpSalGrade(7839, @empSalGrade);
SELECT @empSalGrade;
```

And similarly for the second procedure.

f. setShippingDays

- **Purpose:**

Determines shipping days based on the department's location.

- **How It Works:**

It retrieves the location from the `dept` table using the department number and then uses a CASE statement to set the shipping days.

- **Usage:**

```
sql

CALL setShippingDays(20, @custShippingDays);
SELECT @custShippingDays;
```

g. initDB

- **Purpose:**
Resets the database by deleting all records from the `emp`, `dept`, and `salgrade` tables.
- **Usage:**

```
sql

CALL initDB();
```

3. Key Takeaways

- **Modular Logic:**
Stored procedures allow you to package logic so that you can call the same code repeatedly without rewriting queries.
- **Parameter Types:**
 - **IN:** Passes values to the procedure.
 - **OUT:** Returns values from the procedure.
 - **INOUT:** Both receives and returns a value, allowing for operations like incrementing a counter.
- **Control Structures:**
The use of conditional logic (IF/ELSEIF and CASE) within procedures shows how you can embed decision-making directly in SQL.
- **Data Management:**
The `initDB` procedure is a simple way to clear data and reset tables, which can be particularly useful during testing.

This script not only creates tables and relationships but also provides a hands-on example of using stored procedures for data retrieval, manipulation, and control flow in MySQL. If you have further questions or need more examples on how to extend or modify these procedures, feel free to ask!



Procedures

1. Write a SQL procedure to retrieve all employees with their department names.

```
+-----+-----+-----+-----+
| empno | ename  | job    | dname  |
+-----+-----+-----+-----+
```

2. Write a SQL procedure to count employees in each department.

```
+-----+-----+
|  dname   | emp_count |
+-----+-----+
```

3. Write a SQL procedure to list departments with no employees.

```
+-----+-----+
|  dname   | location |
+-----+-----+
```

4. Write a SQL procedure to find employees with duplicate names.

```
+-----+-----+
|  ename   | name_count |
+-----+-----+
```

5. Write a SQL procedure to find employees earning above 2500 and return their count.

****Procedure:**** (IN parameter: min_salary = 2500)

CALL GetHighEarners(@count); SELECT @count;

```
+-----+-----+-----+
|  ename  | sal   | deptno |
+-----+-----+-----+
```

```
+-----+
| @count |
+-----+
```

6. Write a SQL procedure to list employees and their managers' names and count employees with no manager.

****Procedure:**** (No IN parameters, only OUT)

```
+-----+-----+-----+
| employee | job      | manager |
+-----+-----+-----+
```

```
+-----+
| @no_mgr |
+-----+
```

7. Write a SQL procedure to calculate the total salary expenditure by department and return the highest total.

****Procedure:**** (No IN parameters, only OUT)

CALL TotalSalaryByDept(@max); SELECT @max;

```
+-----+-----+
|  dname   | total_salary |
+-----+-----+
```

```
+-----+
| @max     |
+-----+
```

8. Write a SQL procedure to find employees hired before '1995-01-01' and return their average salary.

****Procedure:**** (IN parameter: cutoff_date = '1995-01-01')

****Execution Command:**** CALL GetEmpByHireDate(@avg); SELECT @avg;

-- Result of SELECT in procedure:

```
+-----+-----+-----+
| empno | ename | hiredate |
+-----+-----+-----+
```

-- Result of SELECT @avg:

```
+-----+
| @avg |
+-----+
```

9. Write a SQL procedure to assign salary grades to employees and return the highest grade assigned.

****Procedure:**** (No IN parameters, only OUT)

****Execution Command:**** CALL AssignSalaryGrades(@max); SELECT @max;

-- Result of SELECT in procedure:

```
+-----+-----+-----+
| ename | sal | grade |
+-----+-----+-----+
```

-- Result of SELECT @max:

```
+-----+
| @max |
+-----+
```

10. Write a SQL procedure to find the highest-paid employee in each department and return the overall highest salary.

****Procedure:**** (No IN parameters, only OUT)

****Execution Command:**** CALL GetTopEarnerByDept(@top); SELECT @top;

-- Result of SELECT in procedure:

```
+-----+-----+-----+
| dname | ename | max_salary |
+-----+-----+-----+
```

-- Result of SELECT @top:

```
+-----+
| @top |
+-----+
```

11. Write a SQL procedure to update employee commission to 1000 for 'SALESMAN' and track the number of updates.

****Procedure:**** (IN parameters: job_type = 'SALESMAN', new_comm = 1000)

****Execution Command:**** CALL UpdateCommByJob(@count); SELECT @count;

-- Result of SELECT in procedure:

```
+-----+-----+-----+-----+
| empno | ename  | job   | comm  |
+-----+-----+-----+-----+
```

-- Result of SELECT @count:

```
+-----+
| @count |
+-----+
```

12. Write a SQL procedure to calculate average salary by job title and modify 'CLERK' average.

****Procedure:**** (IN parameter: target_job = 'CLERK')

****Execution Command:**** SET @avg = 0; CALL AvgSalaryByJob(@avg); SELECT @avg;

-- Result of SELECT in procedure:

```
+-----+-----+
| job   | avg_salary |
+-----+-----+
```

-- Result of SELECT @avg:

```
+-----+
| @avg  |
+-----+
```

13. Write a SQL procedure to increase salaries by 10% for department 20 and track the total increase.

****Procedure:**** (IN parameters: dept_num = 20, pct = 10)

****Execution Command:**** CALL IncreaseSalByDept(@increase); SELECT @increase;

-- Result of SELECT in procedure:

```
+-----+-----+-----+
| empno | ename  | sal   |
+-----+-----+-----+
```

-- Result of SELECT @increase:

```
+-----+
| @increase |
+-----+
```

14. Write a SQL procedure to find employees with no commission in grade 4 and count them.

****Procedure:**** (IN parameter: grade_num = 4)

****Execution Command:**** CALL GetNoCommByGrade(@count); SELECT @count;

-- Result of SELECT in procedure:

```
+-----+-----+-----+
| ename | sal   | grade |
+-----+-----+-----+
```

-- Result of SELECT @count:

```
+-----+
```

```
| @count |
```

```
+-----+
```

15. Write a SQL procedure to list employees reporting to manager 7698 and modify their count.

****Procedure:**** (IN parameter: mgr_id = 7698)

****Execution Command:**** SET @count = 0; CALL GetSubordinates(@count); SELECT @count;

-- Result of SELECT in procedure:

```
+-----+-----+-----+
```

```
| empno | ename | job |
```

```
+-----+-----+-----+
```

-- Result of SELECT @count:

```
+-----+
```

```
| @count |
```

```
+-----+
```

16. Write a SQL procedure to find departments with above-average employee count and return the average.

****Procedure:**** (No IN parameters, only OUT)

****Execution Command:**** CALL AboveAvgEmpDepts(@avg); SELECT @avg;

-- Result of SELECT in procedure:

```
+-----+-----+
```

```
| dname | emp_count |
```

```
+-----+-----+
```

-- Result of SELECT @avg:

```
+-----+
```

```
| @avg |
```

```
+-----+
```

17. Write a SQL procedure to calculate total commission by department and return the maximum commission.

****Procedure:**** (No IN parameters, only OUT)

****Execution Command:**** CALL TotalCommByDept(@max); SELECT @max;

-- Result of SELECT in procedure:

```
+-----+-----+
```

```
| dname | total_comm |
```

```
+-----+-----+
```

-- Result of SELECT @max:

```
+-----+
```

```
| @max |
```

```
+-----+
```

18. Write a SQL procedure to find employees hired in the same year as employee 7499 and count

them.

****Procedure:**** (IN parameter: emp_id = 7499)

****Execution Command:**** CALL SameHireYear(@count); SELECT @count;

-- Result of SELECT in procedure:

empno	ename	hiredate

-- Result of SELECT @count:

@count

19. Write a SQL procedure to rank employees by salary within their department and return the highest rank count.

****Procedure:**** (No IN parameters, only OUT)

****Execution Command:**** CALL RankEmpBySalary(@max); SELECT @max;

-- Result of SELECT in procedure:

ename	sal	dname	salary_rank

-- Result of SELECT @max:

@max

20. Write a SQL procedure to transfer employees from dept 30 to dept 40, increase salaries by 500, and track count.

****Procedure:**** (IN parameters: old_dept = 30, new_dept = 40, sal_increase = 500)

****Execution Command:**** CALL TransferAndUpdateSalary(@count); SELECT @count;

-- Result of SELECT in procedure:

empno	ename	sal	deptno

-- Result of SELECT @count:

@count

Below is one complete SQL script containing all 20 stored procedures. You can run these in your MySQL "test" database. Each procedure is commented to explain its purpose and how it meets the requirements.

```
sql

USE test;

-- =====
-- 1. Retrieve all employees with their department names.
-- =====
DROP PROCEDURE IF EXISTS GetEmpDept;
DELIMITER $$
CREATE PROCEDURE GetEmpDept()
BEGIN
    SELECT e.empno, e.ename, e.job, d.dname
    FROM emp e
    INNER JOIN dept d ON e.deptno = d.deptno;
END$$
DELIMITER ;

-- =====
-- 2. Count employees in each department.
-- =====
DROP PROCEDURE IF EXISTS CountEmpByDept;
DELIMITER $$
CREATE PROCEDURE CountEmpByDept()
BEGIN
    SELECT d.dname, COUNT(e.empno) AS emp_count
    FROM dept d
    LEFT JOIN emp e ON d.deptno = e.deptno
    GROUP BY d.dname;
END$$
DELIMITER ;

-- =====
-- 3. List departments with no employees.
-- =====
DROP PROCEDURE IF EXISTS DeptNoEmp;
DELIMITER $$
CREATE PROCEDURE DeptNoEmp()
BEGIN
    SELECT d.dname, d.location
    FROM dept d
    LEFT JOIN emp e ON d.deptno = e.deptno
    WHERE e.empno IS NULL;
END$$
DELIMITER ;

-- =====
-- 4. Find employees with duplicate names.
-- =====
DROP PROCEDURE IF EXISTS GetDuplicateNames;
DELIMITER $$
CREATE PROCEDURE GetDuplicateNames()
BEGIN
    SELECT ename, COUNT(*) AS name_count
    FROM emp
    GROUP BY ename
    HAVING COUNT(*) > 1;
END$$
DELIMITER ;

-- =====
-- 5. Find employees earning above 2500 and return their count.
-- (Uses a fixed threshold of 2500.)
-- =====
DROP PROCEDURE IF EXISTS GetHighEarners;
DELIMITER $$
CREATE PROCEDURE GetHighEarners(OUT high_count INT)
BEGIN
```

```
-- List employees with salary > 2500.
SELECT ename, sal, deptno
FROM emp
WHERE sal > 2500;

-- Set output parameter to the count.
SELECT COUNT(*) INTO high_count FROM emp WHERE sal > 2500;
END$$
DELIMITER ;

-- =====
-- 6. List employees and their managers' names and count employees with no manager.
-- -----
DROP PROCEDURE IF EXISTS ListEmpWithMgr;
DELIMITER $$
CREATE PROCEDURE ListEmpWithMgr(OUT no_mgr INT)
BEGIN
    -- List employees with their manager's name.
    SELECT e.ename AS employee, e.job, m.ename AS manager
    FROM emp e
    LEFT JOIN emp m ON e.mgr = m.empno;

    -- Count employees with no manager.
    SELECT COUNT(*) INTO no_mgr FROM emp WHERE mgr IS NULL;
END$$
DELIMITER ;

-- =====
-- 7. Calculate total salary expenditure by department and return the highest total.
-- -----
DROP PROCEDURE IF EXISTS TotalSalaryByDept;
DELIMITER $$
CREATE PROCEDURE TotalSalaryByDept(OUT max_total DECIMAL(10,2))
BEGIN
    -- Create a temporary table with each department's total salary.
    CREATE TEMPORARY TABLE dept_salary AS
    SELECT d.dname, SUM(e.sal) AS total_salary
    FROM dept d
    LEFT JOIN emp e ON d.deptno = e.deptno
    GROUP BY d.dname;

    -- Display each department's total.
    SELECT * FROM dept_salary;

    -- Return the highest total salary.
    SELECT MAX(total_salary) INTO max_total FROM dept_salary;

    DROP TEMPORARY TABLE dept_salary;
END$$
DELIMITER ;

-- =====
-- 8. Find employees hired before a given cutoff date and return their average salary.
-- (IN parameter: cutoff_date, e.g. '1995-01-01')
-- -----
DROP PROCEDURE IF EXISTS GetEmpByHireDate;
DELIMITER $$
CREATE PROCEDURE GetEmpByHireDate(IN cutoff_date DATE, OUT avg_sal DECIMAL(10,2))
BEGIN
    -- List employees hired before the cutoff date.
    SELECT empno, ename, hiredate
    FROM emp
    WHERE hiredate < cutoff_date;

    -- Calculate the average salary.
    SELECT AVG(sal) INTO avg_sal FROM emp WHERE hiredate < cutoff_date;
END$$
DELIMITER ;

-- =====
-- 9. Assign salary grades to employees and return the highest grade assigned.
-- (Grade rules: sal > 3000 = 'HIGH', >1000 = 'MEDIUM', otherwise 'LOW')
-- -----
DROP PROCEDURE IF EXISTS AssignSalaryGrades;
DELIMITER $$
```

```

CREATE PROCEDURE AssignSalaryGrades(OUT max_grade VARCHAR(10))
BEGIN
    -- Build a temporary table with employee names, salary, and grade.
    CREATE TEMPORARY TABLE emp_grades AS
        SELECT ename, sal,
            CASE
                WHEN sal > 3000 THEN 'HIGH'
                WHEN sal > 1000 THEN 'MEDIUM'
                ELSE 'LOW'
            END AS grade
        FROM emp;

    -- Display the grade assignment.
    SELECT * FROM emp_grades;

    -- Determine the highest grade (using business logic: HIGH > MEDIUM > LOW).
    IF EXISTS (SELECT 1 FROM emp_grades WHERE grade = 'HIGH') THEN
        SET max_grade = 'HIGH';
    ELSEIF EXISTS (SELECT 1 FROM emp_grades WHERE grade = 'MEDIUM') THEN
        SET max_grade = 'MEDIUM';
    ELSE
        SET max_grade = 'LOW';
    END IF;

    DROP TEMPORARY TABLE emp_grades;
END$$
DELIMITER ;

-- =====
-- 10. Find the highest-paid employee in each department and return the overall highest salary.
-- =====
DROP PROCEDURE IF EXISTS GetTopEarnerByDept;
DELIMITER $$
CREATE PROCEDURE GetTopEarnerByDept(OUT overall_max DECIMAL(10,2))
BEGIN
    -- Create a temporary table with the top earner for each department.
    CREATE TEMPORARY TABLE dept_top AS
        SELECT d.dname, e.ename, e.sal AS max_salary
        FROM dept d
        JOIN emp e ON d.deptno = e.deptno
        WHERE e.sal = (
            SELECT MAX(sal) FROM emp WHERE deptno = d.deptno
        );

    -- Show each department's top earner.
    SELECT * FROM dept_top;

    -- Return the overall highest salary.
    SELECT MAX(max_salary) INTO overall_max FROM dept_top;

    DROP TEMPORARY TABLE dept_top;
END$$
DELIMITER ;

-- =====
-- 11. Update employee commission to 1000 for a given job type and track the number of updates.
--     (IN parameters: job_type, new_comm)
-- =====
DROP PROCEDURE IF EXISTS UpdateCommByJob;
DELIMITER $$
CREATE PROCEDURE UpdateCommByJob(IN job_type VARCHAR(50), IN new_comm DECIMAL(10,2), OUT update_cou
BEGIN
    UPDATE emp
    SET comm = new_comm
    WHERE job = job_type;

    SET update_count = ROW_COUNT();

    SELECT empno, ename, job, comm
    FROM emp
    WHERE job = job_type;
END$$
DELIMITER ;

-- =====

```

```
-- 12. Calculate average salary by job title and modify the average for a target job.
--      (IN parameter: target_job, e.g. 'CLERK'; here we increase its average by 10%)
-----
DROP PROCEDURE IF EXISTS AvgSalaryByJob;
DELIMITER $$
CREATE PROCEDURE AvgSalaryByJob(IN target_job VARCHAR(50), OUT modified_avg DECIMAL(10,2))
BEGIN
    -- Show average salary for each job.
    SELECT job, AVG(sal) AS avg_salary
    FROM emp
    GROUP BY job;

    -- Compute the average salary for the target job.
    SELECT AVG(sal) INTO modified_avg
    FROM emp
    WHERE job = target_job;

    -- Modify the average (for example, increase by 10%).
    SET modified_avg = modified_avg * 1.1;
END$$
DELIMITER ;

-- =====
-- 13. Increase salaries by a given percentage for a department and track the total increase.
--      (IN parameters: dept_num, pct (percentage); here total increase is approximated.
--      Note: We compute the original sum from updated records.)
-----
DROP PROCEDURE IF EXISTS IncreaseSalByDept;
DELIMITER $$
CREATE PROCEDURE IncreaseSalByDept(IN dept_num INT, IN pct DECIMAL(5,2), OUT total_increase DECIMAL(10,2))
BEGIN
    -- Before update, get the original sum of salaries for the department.
    DECLARE orig_sum DECIMAL(10,2);
    SELECT SUM(sal) INTO orig_sum FROM emp WHERE deptno = dept_num;

    -- Update salaries.
    UPDATE emp
    SET sal = sal * (1 + pct/100)
    WHERE deptno = dept_num;

    -- Calculate the total increase.
    SET total_increase = (pct/100) * orig_sum;

    SELECT empno, ename, sal, deptno FROM emp WHERE deptno = dept_num;
END$$
DELIMITER ;

-- =====
-- 14. Find employees with no commission in a specific grade and count them.
--      (IN parameter: grade_num; join emp with salgrade to check salary boundaries)
-----
DROP PROCEDURE IF EXISTS GetNoCommByGrade;
DELIMITER $$
CREATE PROCEDURE GetNoCommByGrade(IN grade_num INT, OUT count_no_comm INT)
BEGIN
    SELECT e.ename, e.sal, s.grade
    FROM emp e
    JOIN salgrade s ON e.sal BETWEEN s.losal AND s.hisal
    WHERE s.grade = grade_num AND (e.comm IS NULL OR e.comm = 0);

    SELECT COUNT(*) INTO count_no_comm
    FROM emp e
    JOIN salgrade s ON e.sal BETWEEN s.losal AND s.hisal
    WHERE s.grade = grade_num AND (e.comm IS NULL OR e.comm = 0);
END$$
DELIMITER ;

-- =====
-- 15. List employees reporting to a specific manager and return their count.
--      (IN parameter: mgr_id; for example, mgr_id = 7698)
-----
DROP PROCEDURE IF EXISTS GetSubordinates;
DELIMITER $$
CREATE PROCEDURE GetSubordinates(IN mgr_id INT, OUT sub_count INT)
BEGIN
```

```

SELECT empno, ename, job
FROM emp
WHERE mgr = mgr_id;

SELECT COUNT(*) INTO sub_count FROM emp WHERE mgr = mgr_id;
END$$
DELIMITER ;

-- =====
-- 16. Find departments with above-average employee count and return the average count.
-- (No IN parameters, only OUT)
-- -----
DROP PROCEDURE IF EXISTS AboveAvgEmpDepts;
DELIMITER $$
CREATE PROCEDURE AboveAvgEmpDepts(OUT avg_count DECIMAL(10,2))
BEGIN
    -- Calculate average employee count per department.
    SELECT AVG(emp_count) INTO avg_count FROM (
        SELECT COUNT(e.empno) AS emp_count
        FROM dept d
        LEFT JOIN emp e ON d.deptno = e.deptno
        GROUP BY d.deptno
    ) AS counts;

    -- List departments exceeding the average count.
    SELECT d.dname, COUNT(e.empno) AS emp_count
    FROM dept d
    LEFT JOIN emp e ON d.deptno = e.deptno
    GROUP BY d.dname
    HAVING COUNT(e.empno) > avg_count;
END$$
DELIMITER ;

-- =====
-- 17. Calculate total commission by department and return the maximum commission.
-- (No IN parameters, only OUT)
-- -----
DROP PROCEDURE IF EXISTS TotalCommByDept;
DELIMITER $$
CREATE PROCEDURE TotalCommByDept(OUT max_comm DECIMAL(10,2))
BEGIN
    CREATE TEMPORARY TABLE dept_comm AS
    SELECT d.dname, SUM(e.comm) AS total_comm
    FROM dept d
    LEFT JOIN emp e ON d.deptno = e.deptno
    GROUP BY d.dname;

    SELECT * FROM dept_comm;

    SELECT MAX(total_comm) INTO max_comm FROM dept_comm;

    DROP TEMPORARY TABLE dept_comm;
END$$
DELIMITER ;

-- =====
-- 18. Find employees hired in the same year as a given employee and count them.
-- (IN parameter: emp_id; e.g. 7499)
-- -----
DROP PROCEDURE IF EXISTS SameHireYear;
DELIMITER $$
CREATE PROCEDURE SameHireYear(IN emp_id INT, OUT count_same INT)
BEGIN
    DECLARE target_year YEAR;
    SELECT YEAR(hiredate) INTO target_year FROM emp WHERE empno = emp_id;

    SELECT empno, ename, hiredate
    FROM emp
    WHERE YEAR(hiredate) = target_year;

    SELECT COUNT(*) INTO count_same FROM emp WHERE YEAR(hiredate) = target_year;
END$$
DELIMITER ;

-- =====

```

```
-- 19. Rank employees by salary within their department and return the highest rank count.
-- (No IN parameters, only OUT)
-- (This uses MySQL 8 window functions.)
-----
DROP PROCEDURE IF EXISTS RankEmpBySalary;
DELIMITER $$
CREATE PROCEDURE RankEmpBySalary(OUT max_rank INT)
BEGIN
    WITH ranked AS (
        SELECT e.ename, e.sal, d.dname,
               RANK() OVER (PARTITION BY e.deptno ORDER BY e.sal DESC) AS salary_rank
        FROM emp e
        JOIN dept d ON e.deptno = d.deptno
    )
    SELECT * FROM ranked;

    -- Determine the maximum rank across all departments.
    SELECT MAX(salary_rank) INTO max_rank FROM (
        SELECT RANK() OVER (PARTITION BY e.deptno ORDER BY e.sal DESC) AS salary_rank
        FROM emp e
    ) AS r;
END$$
DELIMITER ;

-- =====
-- 20. Transfer employees from one department to another, increase salaries by a fixed amount, and
-- (IN parameters: old_dept, new_dept, sal_increase)
-----
DROP PROCEDURE IF EXISTS TransferAndUpdateSalary;
DELIMITER $$
CREATE PROCEDURE TransferAndUpdateSalary(IN old_dept INT, IN new_dept INT, IN sal_increase DECIMAL(10,2))
BEGIN
    UPDATE emp
    SET deptno = new_dept,
        sal = sal + sal_increase
    WHERE deptno = old_dept;

    SET transfer_count = ROW_COUNT();

    SELECT empno, ename, sal, deptno FROM emp WHERE deptno = new_dept;
END$$
DELIMITER ;

-- =====
-- To view all stored procedures in the test database:
SHOW PROCEDURE STATUS WHERE Db = 'test';
```

Usage Examples

1. Get Employees with Departments:

```
sql

CALL GetEmpDept();
```

2. Count Employees by Department:

```
sql

CALL CountEmpByDept();
```

3. List Departments with No Employees:

```
sql  
  
CALL DeptNoEmp();
```

4. Find Duplicate Employee Names:

```
sql  
  
CALL GetDuplicateNames();
```

5. Find High Earners (above 2500) and get count:

```
sql  
  
CALL GetHighEarners(@count);  
SELECT @count;
```

6. List Employees with Their Managers and Count No Manager:

```
sql  
  
CALL ListEmpWithMgr(@no_mgr);  
SELECT @no_mgr;
```

7. Get Total Salary by Department and Overall Maximum:

```
sql  
  
CALL TotalSalaryByDept(@max_total);  
SELECT @max_total;
```

8. Employees Hired Before a Date and Average Salary:

```
sql  
  
CALL GetEmpByHireDate('1995-01-01', @avg_sal);  
SELECT @avg_sal;
```

9. Assign Salary Grades and Return Highest Grade Assigned:

```
sql  
  
CALL AssignSalaryGrades(@max_grade);  
SELECT @max_grade;
```

10. Get Top Earner by Department and Overall Highest Salary:

```
sql  
  
CALL GetTopEarnerByDept(@overall_max);  
SELECT @overall_max;
```

11. Update Commission for SALESMAN and Get Update Count:

```
sql
```

```
CALL UpdateCommByJob('SALESMAN', 1000, @update_count);
SELECT @update_count;
```

12. Average Salary by Job Title with Modified 'CLERK' Average:

```
sql
```

```
SET @avg = 0;
CALL AvgSalaryByJob('CLERK', @avg);
SELECT @avg;
```

13. Increase Salaries by 10% for Department 20 and Get Total Increase:

```
sql
```

```
CALL IncreaseSalByDept(20, 10, @total_increase);
SELECT @total_increase;
```

14. Find Employees with No Commission in Grade 4 and Count Them:

```
sql
```

```
CALL GetNoCommByGrade(4, @count_no_comm);
SELECT @count_no_comm;
```

15. List Subordinates of Manager 7698 and Count Them:

```
sql
```

```
CALL GetSubordinates(7698, @sub_count);
SELECT @sub_count;
```

16. List Departments with Above-Average Employee Count and Return the Average:

```
sql
```

```
CALL AboveAvgEmpDepts(@avg_count);
SELECT @avg_count;
```

17. Calculate Total Commission by Department and Return the Maximum:

```
sql
```

```
CALL TotalCommByDept(@max_comm);
SELECT @max_comm;
```

18. Find Employees Hired in the Same Year as Employee 7499 and Count Them:

```
sql
```

```
CALL SameHireYear(7499, @count_same);
SELECT @count_same;
```

19. Rank Employees by Salary and Return the Highest Rank:


```
sql
```

```
CALL RankEmpBySalary(@max_rank);  
SELECT @max_rank;
```

20. Transfer Employees from Dept 30 to Dept 40, Increase Salary by 500, and Get Count:

```
sql
```

```
CALL TransferAndUpdateSalary(30, 40, 500, @transfer_count);  
SELECT @transfer_count;
```

Each procedure encapsulates a piece of business logic or reporting requirement. Adjust parameters or logic as needed for your specific data and use case. Enjoy building with MySQL procedures!

ChatGPT can make mistakes. Check important info.